

# Nested Varying Effects

## General Principles

To model hierarchies where groups are themselves nested within larger units (e.g., students within classes, which are within schools), we use *Nested Varying Effects*. This allows the model to share information across different levels of the hierarchy, improving estimates through multilevel pooling.

## Note

### Note

- We have the same considerations as for [Varying Intercepts](#) and [Varying Slopes](#).
- In a nested model, a parameter at one level (e.g., a group-level intercept) becomes the mean for the distribution of parameters at a lower level (e.g., individual-level intercepts).
- This structure is often represented using multiple indices:  $j$  for the group and  $k(j)$  for the super-group (e.g., region) that group  $j$  belongs to.
- To capture the correlation between multiple varying effects (e.g., intercept and slope) at any level of the hierarchy, we use a **Multivariate Normal (MVN)** distribution.
- Non-centered parameterization is highly recommended for nested models to avoid “funnel” geometries that can hinder MCMC sampling.

## Example

Below is an example of a nested model with both **varying intercepts** and **varying slopes**. We model the outcome  $y$  for individuals in **groups** nested within **regions**. The relationship between  $x$  and  $y$  varies at both levels.

## Python (Raw MVN)

```
from BI import bi
import jax.numpy as jnp
import jax
import numpy as np

# Setup device-----
m = bi(platform='cpu')

# Simulated Data -----
np.random.seed(42)
N_regions = 5
N_groups_per_region = 4
N_groups = N_regions * N_groups_per_region
N_obs_per_group = 20
N = N_groups * N_obs_per_group

# Region level (Top level) parameters
mu_a_region, mu_b_region = 5.0, -1.0
sigma_a_region, sigma_b_region = 1.0, 0.5
rho_region = -0.5

# Group level (Nested level) parameters
sigma_a_group, sigma_b_group = 0.5, 0.2
rho_group = 0.3
sigma_obs = 0.5

# Generate region effects
mu_region = np.array([mu_a_region, mu_b_region])
sigmas_region = np.array([sigma_a_region, sigma_b_region])
Rho_region = np.array([[1, rho_region], [rho_region, 1]])
Sigma_region = np.outer(sigmas_region, sigmas_region) * Rho_region
region_effects = np.random.multivariate_normal(mu_region, Sigma_region, size=N_regions)

# Generate group effects nested in regions
group_to_region = np.repeat(np.arange(N_regions), N_groups_per_region)
sigmas_group = np.array([sigma_a_group, sigma_b_group])
Rho_group = np.array([[1, rho_group], [rho_group, 1]])
Sigma_group = np.outer(sigmas_group, sigmas_group) * Rho_group

group_effects = np.zeros((N_groups, 2))
```

```

for g in range(N_groups):
    reg = group_to_region[g]
    group_effects[g] = np.random.multivariate_normal(region_effects[reg], Sigma_group)

# Generate observations
group_id = np.repeat(np.arange(N_groups), N_obs_per_group)
x = np.random.normal(0, 1, size=N) # 1D
a_g = group_effects[:, 0]
b_g = group_effects[:, 1]
mu = a_g[group_id] + b_g[group_id] * x
y = np.random.normal(mu, sigma_obs) # 1D

m.data_on_model = {
    "group_id": group_id.astype(np.int32),
    "x": x.astype(np.float32),
    "y": y.astype(np.float32),
    "group_to_region": group_to_region.astype(np.int32),
    "N_regions": N_regions,
}

# Define model -----
def model_nested(group_id, x, y, group_to_region, N_regions):
    sigma = m.dist.exponential(1, name='sigma')

    # 1. Region level
    mu_a_reg = m.dist.normal(5, 2, name='mu_a_reg')
    mu_b_reg = m.dist.normal(-1, 1, name='mu_b_reg')
    sigma_reg = m.dist.exponential(1, shape=(2,), name='sigma_reg')
    Rho_reg = m.dist.lkj(2, 2, name='Rho_reg')
    cov_reg = jnp.outer(sigma_reg, sigma_reg) * Rho_reg

    region_effects = m.dist.multivariate_normal(
        jnp.stack([mu_a_reg, mu_b_reg]),
        cov_reg,
        shape=(N_regions,),
        name='region_effects'
    )

    # 2. Group level
    sigma_grp = m.dist.exponential(1, shape=(2,), name='sigma_grp')
    Rho_grp = m.dist.lkj(2, 2, name='Rho_grp')
    cov_grp = jnp.outer(sigma_grp, sigma_grp) * Rho_grp

```

```

# Nested mean
mu_grp = region_effects[group_to_region]

group_effects = m.dist.multivariate_normal(
    mu_grp,
    cov_grp,
    name='group_effects'
)

a_g_est = group_effects[:, 0]
b_g_est = group_effects[:, 1]

# Likelihood
mu_est = a_g_est[group_id] + b_g_est[group_id] * x
m.dist.normal(mu_est, sigma, obs=y)

# Run sampler -----
m.fit(model_nested, num_samples=1000, num_warmup=500, num_chains=1, progress_bar=False)

# Diagnostic -----
m.summary()

```

jax.local\_device\_count 16

arviz - WARNING - Shape validation failed: input\_shape: (1, 1000), minimum\_shape: (chains=2, C:\Users\Sosa\AppData\Local\Programs\Python\Python313\Lib\site-packages\arviz\stats\diagnost.

invalid value encountered in scalar divide

C:\Users\Sosa\AppData\Local\Programs\Python\Python313\Lib\site-packages\arviz\stats\diagnost.

invalid value encountered in scalar divide

|               | mean | sd   | hdi_5.5% | hdi_94.5% | mcse_mean | mcse_sd | ess_bulk | ess_tail | r_hat |
|---------------|------|------|----------|-----------|-----------|---------|----------|----------|-------|
| Rho_grp[0, 0] | 1.00 | 0.00 | 1.00     | 1.00      | 0.00      | NaN     | 1000.00  | 1000.00  | NaN   |
| Rho_grp[0, 1] | 0.42 | 0.23 | 0.06     | 0.76      | 0.01      | 0.01    | 787.43   | 788.58   | NaN   |
| Rho_grp[1, 0] | 0.42 | 0.23 | 0.06     | 0.76      | 0.01      | 0.01    | 787.43   | 788.58   | NaN   |
| Rho_grp[1, 1] | 1.00 | 0.00 | 1.00     | 1.00      | 0.00      | 0.00    | 985.81   | 865.91   | NaN   |
| Rho_reg[0, 0] | 1.00 | 0.00 | 1.00     | 1.00      | 0.00      | NaN     | 1000.00  | 1000.00  | NaN   |

|              | mean | sd   | hdi_5.5% | hdi_94.5% | mcse_mean | mcse_sd | ess_bulk | ess_tail | r_hat |
|--------------|------|------|----------|-----------|-----------|---------|----------|----------|-------|
| ...          | ...  | ...  | ...      | ...       | ...       | ...     | ...      | ...      | ...   |
| sigma        | 0.49 | 0.02 | 0.46     | 0.52      | 0.00      | 0.00    | 1618.21  | 764.96   | NaN   |
| sigma_grp[0] | 0.42 | 0.09 | 0.29     | 0.55      | 0.00      | 0.00    | 976.50   | 813.28   | NaN   |
| sigma_grp[1] | 0.25 | 0.06 | 0.15     | 0.33      | 0.00      | 0.00    | 590.01   | 593.31   | NaN   |
| sigma_reg[0] | 1.00 | 0.38 | 0.50     | 1.51      | 0.01      | 0.02    | 1012.78  | 788.33   | NaN   |
| sigma_reg[1] | 0.47 | 0.26 | 0.16     | 0.76      | 0.01      | 0.02    | 701.74   | 676.84   | NaN   |

## Python (Hierarchical Normal)

```

from BI import bi, jnp
import jax
import numpy as np
# Setup device-----
m = bi(platform='cpu')

# Simulated Data -----
np.random.seed(42)
N_regions = 5
N_groups_per_region = 4
N_groups = N_regions * N_groups_per_region
N_obs_per_group = 20
N = N_groups * N_obs_per_group

# Simulation parameters
mu_a_region, mu_b_region = 5.0, -1.0
sigma_a_region, sigma_b_region = 1.0, 0.5
rho_region = -0.5
sigma_a_group, sigma_b_group = 0.5, 0.2
rho_group = 0.3
sigma_obs = 0.5

# Generate region effects
mu_region = np.array([mu_a_region, mu_b_region])
sigmas_region = np.array([sigma_a_region, sigma_b_region])
Rho_region = np.array([[1, rho_region], [rho_region, 1]])
Sigma_region = np.outer(sigmas_region, sigmas_region) * Rho_region
region_effects = np.random.multivariate_normal(mu_region, Sigma_region, size=N_regions)

# Generate group effects nested in regions

```

```

group_to_region = np.repeat(np.arange(N_regions), N_groups_per_region)
sigmas_group = np.array([sigma_a_group, sigma_b_group])
Rho_group = np.array([[1, rho_group], [rho_group, 1]])
Sigma_group = np.outer(sigmas_group, sigmas_group) * Rho_group
group_effects = np.zeros((N_groups, 2))
for g in range(N_groups):
    reg = group_to_region[g]
    group_effects[g] = np.random.multivariate_normal(region_effects[reg], Sigma_group)

# Generate observations
group_id = np.repeat(np.arange(N_groups), N_obs_per_group)
x = np.random.normal(0, 1, size=N).astype(np.float32)
y = np.random.normal(0, 1, size=N).astype(np.float32)

m.data_on_model = {
    "group_id": group_id.astype(np.int32),
    "group_to_region": group_to_region.astype(np.int32),
    "x": x,
    "y": y,
}

# Define model -----
def model_hierarchical(group_id, group_to_region, x, y):
    sigma = m.dist.exponential(1, name='sigma')

    # 1. Region Level
    a_reg = m.dist.normal(5, 2, shape=(5,), name='a_reg')
    b_reg = m.dist.normal(-1, 1, shape=(5,), name='b_reg')

    # 2. Group Level (Nested)
    # Means are indexed from region level
    a_grp = m.dist.normal(a_reg[group_to_region], 0.5, shape=(20,), name='a_grp')
    b_grp = m.dist.normal(b_reg[group_to_region], 0.2, shape=(20,), name='b_grp')

    # 3. Likelihood
    mu = a_grp[group_id] + b_grp[group_id] * x
    m.dist.normal(mu, sigma, obs=y)

# Run sampler -----
m.fit(model_hierarchical, num_samples=1000, num_warmup=500, num_chains=1, progress_bar=False)
m.summary()

```

jax.local\_device\_count 16

arviz - WARNING - Shape validation failed: input\_shape: (1, 1000), minimum\_shape: (chains=2,

|           | mean  | sd   | hdi_5.5% | hdi_94.5% | mcse_mean | mcse_sd | ess_bulk | ess_tail | r_hat |
|-----------|-------|------|----------|-----------|-----------|---------|----------|----------|-------|
| a_grp[0]  | -0.08 | 0.20 | -0.39    | 0.23      | 0.01      | 0.01    | 1119.30  | 622.62   | NaN   |
| a_grp[1]  | -0.01 | 0.21 | -0.37    | 0.30      | 0.01      | 0.01    | 1401.77  | 817.89   | NaN   |
| a_grp[2]  | -0.12 | 0.20 | -0.47    | 0.15      | 0.01      | 0.01    | 1325.86  | 612.23   | NaN   |
| a_grp[3]  | -0.25 | 0.21 | -0.56    | 0.10      | 0.01      | 0.01    | 1058.34  | 651.24   | NaN   |
| a_grp[4]  | -0.22 | 0.21 | -0.55    | 0.11      | 0.01      | 0.01    | 1122.55  | 598.79   | NaN   |
| a_grp[5]  | -0.03 | 0.22 | -0.36    | 0.32      | 0.01      | 0.01    | 1225.01  | 918.86   | NaN   |
| a_grp[6]  | 0.13  | 0.21 | -0.20    | 0.44      | 0.01      | 0.01    | 1564.94  | 986.53   | NaN   |
| a_grp[7]  | 0.08  | 0.21 | -0.22    | 0.43      | 0.01      | 0.01    | 1146.94  | 879.69   | NaN   |
| a_grp[8]  | 0.36  | 0.20 | 0.03     | 0.68      | 0.01      | 0.01    | 1026.59  | 575.75   | NaN   |
| a_grp[9]  | -0.34 | 0.20 | -0.64    | -0.02     | 0.01      | 0.01    | 1065.07  | 763.01   | NaN   |
| a_grp[10] | 0.05  | 0.21 | -0.27    | 0.38      | 0.01      | 0.01    | 1229.62  | 725.90   | NaN   |
| a_grp[11] | -0.04 | 0.20 | -0.34    | 0.29      | 0.01      | 0.01    | 1228.25  | 765.93   | NaN   |
| a_grp[12] | -0.03 | 0.20 | -0.35    | 0.30      | 0.01      | 0.01    | 1548.30  | 761.37   | NaN   |
| a_grp[13] | -0.03 | 0.24 | -0.39    | 0.34      | 0.01      | 0.01    | 1083.93  | 665.83   | NaN   |
| a_grp[14] | -0.13 | 0.22 | -0.47    | 0.22      | 0.01      | 0.01    | 895.70   | 798.37   | NaN   |
| a_grp[15] | 0.29  | 0.21 | -0.02    | 0.63      | 0.01      | 0.01    | 853.51   | 757.79   | NaN   |
| a_grp[16] | 0.08  | 0.21 | -0.25    | 0.40      | 0.01      | 0.01    | 893.41   | 738.57   | NaN   |
| a_grp[17] | 0.02  | 0.21 | -0.31    | 0.35      | 0.01      | 0.01    | 1257.79  | 685.58   | NaN   |
| a_grp[18] | 0.34  | 0.21 | 0.01     | 0.66      | 0.01      | 0.01    | 997.18   | 771.70   | NaN   |
| a_grp[19] | 0.05  | 0.22 | -0.28    | 0.42      | 0.01      | 0.01    | 1439.33  | 705.49   | NaN   |
| a_reg[0]  | -0.03 | 0.28 | -0.47    | 0.41      | 0.01      | 0.01    | 1325.93  | 695.45   | NaN   |
| a_reg[1]  | 0.07  | 0.27 | -0.36    | 0.48      | 0.01      | 0.01    | 1424.69  | 766.63   | NaN   |
| a_reg[2]  | 0.08  | 0.26 | -0.29    | 0.53      | 0.01      | 0.01    | 1593.36  | 803.20   | NaN   |
| a_reg[3]  | 0.11  | 0.26 | -0.30    | 0.51      | 0.01      | 0.01    | 1011.98  | 767.75   | NaN   |
| a_reg[4]  | 0.21  | 0.27 | -0.20    | 0.63      | 0.01      | 0.01    | 1049.29  | 747.95   | NaN   |
| b_grp[0]  | -0.25 | 0.19 | -0.55    | 0.07      | 0.01      | 0.01    | 753.83   | 678.99   | NaN   |
| b_grp[1]  | -0.41 | 0.17 | -0.67    | -0.14     | 0.01      | 0.00    | 797.09   | 721.12   | NaN   |
| b_grp[2]  | -0.31 | 0.19 | -0.62    | -0.04     | 0.01      | 0.00    | 579.61   | 862.98   | NaN   |
| b_grp[3]  | -0.28 | 0.15 | -0.53    | -0.04     | 0.00      | 0.00    | 990.77   | 847.17   | NaN   |
| b_grp[4]  | -0.07 | 0.18 | -0.36    | 0.20      | 0.01      | 0.01    | 645.65   | 666.39   | NaN   |
| b_grp[5]  | -0.04 | 0.18 | -0.34    | 0.23      | 0.01      | 0.00    | 782.84   | 717.47   | NaN   |
| b_grp[6]  | 0.07  | 0.17 | -0.22    | 0.33      | 0.01      | 0.01    | 628.63   | 672.94   | NaN   |
| b_grp[7]  | 0.04  | 0.16 | -0.21    | 0.30      | 0.01      | 0.01    | 956.89   | 820.82   | NaN   |
| b_grp[8]  | -0.25 | 0.16 | -0.50    | 0.01      | 0.00      | 0.00    | 1154.23  | 786.93   | NaN   |
| b_grp[9]  | -0.20 | 0.17 | -0.45    | 0.07      | 0.01      | 0.00    | 1136.26  | 861.47   | NaN   |
| b_grp[10] | -0.21 | 0.14 | -0.43    | 0.03      | 0.00      | 0.00    | 843.84   | 793.40   | NaN   |

|           | mean  | sd   | hdi_5.5% | hdi_94.5% | mcse_mean | mcse_sd | ess_bulk | ess_tail | r_hat |
|-----------|-------|------|----------|-----------|-----------|---------|----------|----------|-------|
| b_grp[11] | -0.17 | 0.16 | -0.43    | 0.07      | 0.00      | 0.00    | 1094.01  | 676.94   | NaN   |
| b_grp[12] | -0.29 | 0.22 | -0.65    | 0.04      | 0.01      | 0.01    | 542.54   | 576.14   | NaN   |
| b_grp[13] | -0.12 | 0.21 | -0.42    | 0.23      | 0.01      | 0.01    | 503.83   | 715.42   | NaN   |
| b_grp[14] | -0.05 | 0.23 | -0.41    | 0.31      | 0.01      | 0.01    | 548.99   | 715.85   | NaN   |
| b_grp[15] | -0.42 | 0.22 | -0.75    | -0.08     | 0.01      | 0.01    | 588.39   | 705.18   | NaN   |
| b_grp[16] | 0.08  | 0.16 | -0.18    | 0.31      | 0.00      | 0.01    | 1064.12  | 693.98   | NaN   |
| b_grp[17] | -0.12 | 0.17 | -0.39    | 0.14      | 0.01      | 0.01    | 739.80   | 742.55   | NaN   |
| b_grp[18] | -0.03 | 0.16 | -0.30    | 0.20      | 0.01      | 0.00    | 1008.37  | 873.75   | NaN   |
| b_grp[19] | 0.14  | 0.15 | -0.10    | 0.38      | 0.01      | 0.00    | 886.22   | 731.30   | NaN   |
| b_reg[0]  | -0.32 | 0.15 | -0.57    | -0.10     | 0.01      | 0.00    | 492.23   | 639.48   | NaN   |
| b_reg[1]  | -0.01 | 0.15 | -0.24    | 0.25      | 0.01      | 0.00    | 510.54   | 787.98   | NaN   |
| b_reg[2]  | -0.22 | 0.14 | -0.46    | -0.02     | 0.01      | 0.00    | 698.37   | 679.63   | NaN   |
| b_reg[3]  | -0.23 | 0.19 | -0.54    | 0.07      | 0.01      | 0.00    | 353.36   | 529.40   | NaN   |
| b_reg[4]  | 0.01  | 0.14 | -0.20    | 0.23      | 0.01      | 0.00    | 616.95   | 722.53   | NaN   |
| sigma     | 0.98  | 0.03 | 0.92     | 1.03      | 0.00      | 0.00    | 812.75   | 873.70   | NaN   |

## Julia

```
using BayesianInference

# Setup device-----
m = importBI(platform="cpu")

# Define model -----
@BI function model(group_id, region_id, x, y)
    sigma = m.dist.exponential(1, name="sigma")

    # Region Level
    a_reg, b_reg = m.effects.varying_effects(
        N_vars = 1,
        N_group = 5,
        group_id = region_id,
        group_name = "region"
    )

    # Group Level (Nested)
    a_grp, b_grp = m.effects.varying_effects(
        N_vars = 1,
        N_group = 20,
```



```

        group_id = group_id,
        alpha_bar = a_reg,
        beta_bar = b_reg,
        group_name = "group"
    )

    mu = a_grp .+ b_grp .* x
    m.dist.normal(mu, sigma, obs=y)
end

# Run mcmc -----
m.fit(model)
m.summary()

```

## R

```

library(BayesianInference)
jnp = reticulate::import('jax.numpy')

# setup platform-----
m=importBI(platform='cpu')

# Define model -----
model <- function(group_id, region_id, x, y){
  sigma = bi.dist.exponential(1, name='sigma')

  # Region level
  region_veff = m$effects$varying_effects(
    N_vars = 1L,
    N_group = 5L,
    group_id = region_id,
    group_name = 'region'
  )
  a_reg = region_veff[[1]]
  b_reg = region_veff[[2]]

  # Group level (Nested)
  group_veff = m$effects$varying_effects(
    N_vars = 1L,
    N_group = 20L,
    group_id = group_id,

```

```

    alpha_bar = a_reg,
    beta_bar = b_reg,
    group_name = 'group'
  )
  a_grp = group_veff[[1]]
  b_grp = group_veff[[2]]

  mu = a_grp + b_grp * x
  m$dist$normal(mu, sigma, obs=y)
}

# Run MCMC -----
m$fit(model)
m$summary()

```

## Mathematical Details

We model the outcome  $Y_i$  for observation  $i$ . The intercept  $\alpha_j$  and slope  $\beta_j$  for group  $j(i)$  are nested within region  $k(j)$ .

$$Y_i \sim \text{Normal}(\mu_i, \sigma)$$

$$\mu_i = \alpha_{[j(i)]} + \beta_{[j(i)]} X_i$$

The vector of group-level effects follows a Multivariate Normal distribution centered on the region-level effects:

$$\begin{bmatrix} \alpha_j \\ \beta_j \end{bmatrix} \sim \text{MVNormal} \left( \begin{bmatrix} \gamma_{\alpha, [k(j)]} \\ \gamma_{\beta, [k(j)]} \end{bmatrix}, \Sigma_{group} \right)$$

The region-level effects themselves follow their own Multivariate Normal distribution:

$$\begin{bmatrix} \gamma_{\alpha, k} \\ \gamma_{\beta, k} \end{bmatrix} \sim \text{MVNormal} \left( \begin{bmatrix} \bar{\gamma}_{\alpha} \\ \bar{\gamma}_{\beta} \end{bmatrix}, \Sigma_{region} \right)$$

Where:

- $\Sigma = \mathbf{SRS}$  (diagonal matrix of standard deviations  $\mathbf{S}$  and correlation matrix  $\mathbf{R}$ ).
- $\bar{\gamma}_{\alpha}, \bar{\gamma}_{\beta}$  are the global average intercept and slope.
- $\Sigma_{region}, \Sigma_{group}$  are the covariance matrices for each hierarchical level.

## Notes

### Note

- By nesting the **MVN** prior, we allow the model to learn how the correlation between intercepts and slopes persists across different levels of the hierarchy.
- The non-centered parameterization (used in the `m.effects.varying_effects` built-in function) is critical for convergence in these complex models.
- This approach can be generalized to  $N$  levels and  $M$  variables by increasing the dimensionality of the vectors and matrices.

## Reference(s)